Performing CRUD in MYSQL with Express JS

By Pranav Verma

8 mins read Last updated: 2 Feb 2024 201 views

Topics Covered

Overview

Have you ever thought about how the data is stored in a database like MYSQL? How to perform crud operation in MySQL using express.js?

MySQL is an open-source relational database management system (RDBMS) that manages and manipulates data stored in a database using Structured Query Language (SQL). It is one of the world's most popular and commonly used database systems, especially for web applications.

MySQL is used to store and handle huge amounts of data by many major companies and organizations, including Facebook, Twitter, and YouTube. Its scalability, dependability, and ease of use make it an excellent choice for both small and large-scale projects.

CRUD is the collection of fundamental operations that are commonly performed while working with databases while performing CRUD in MYSQL with express.js.

Introduction

CRUD stands for Create, Read, Update, and Delete. It is a set of four fundamental procedures often used while working with databases, especially relational databases such as MySQL, PostgreSQL, and Oracle.

- **Create**: This process involves adding new data to the database. The INSERT statement is commonly used to do this in SQL.
- **Read**: This process involves finding the data from the database. The SELECT statement is commonly used to do this in SQL.
- **Update**: This process involves modifying the database's current data. The UPDATE statement in SQL is often used to do this.
- **Delete**: This process involves removing the data from the database. In SQL, this is done via the DELETE statement.

Install Prerequisite for Express.js

To get started with Express.js performing CRUD in MYSQL, we have to install node.js and npm (Node Package Manager) on our machines. The following are the steps for installing the prerequisites:

- **Nodejs:** Download the latest and most stable version of Node.js for the operating system. Install Node.js in the system.

- Now we will check that Node.js is completely installed in our system or not by running a command in the command prompt or the terminal window:

```
node -v
```

The above command will return the version installed on the machine.

- Npm: npm is the package manager for Node.js which allows users to easily install and manage packages and dependencies.

- Now we will check that npm is completely installed in our system or not by running a command in the command prompt or the terminal window:

```
npm -v
```

The above command will return the version installed in the machine and helps in performing CRUD in MYSQL with Express.js.

Create CRUD Express App

Now, let's discuss how to create a crud express app in the below steps:

- Create a new directory for the project and navigate it in the terminal:

```
mkdir my-express-app
cd my-express-app
```

- Initialising a new Node.js project using npm:

```
npm init
```

- Installing the Express framework, body-parser and MySQL using npm:

```
npm install express body-parser MySQL
```

Here, body-parser is used to parse HTTP request bodies and the MySQL package which is a Nodejs driver for MySQL.

- Creating a new file called index.js.

- Setting up a basic express.js server in index.js.

```
const express = require('express');
const bodyParser = require('body-parser');
const mysql = require('mysql');

const app = express();
const port = 3000;

// Parse the requests of content-type 'application/json'
app.use(bodyParser.json());

// Create the MySQL connection pool
const pool = mysql.createPool ({
```

```
    connectionLimit: 10,
    host: 'localhost',
    user: 'root',
    password: 'password',
    database: 'my_database'
});

// Start a server
app.listen(port, () => {
    console.log(`Server is listening on port ${port}`);
});
```

In the above code, we have created an express.js app, defined a basic route for each CRUD operation and set up the MYSQL connection pool. We will then add the route handlers for each operation in the next steps.

- Adding a route handler for the create operation(INSERT).

```
app.post('/users', (req, res) => {
    const { name, emailaddress } = req.body;

    pool.query('INSERT INTO users (name, emailaddress) VALUES (?, ?)', [
        if (error) {
            console.error(error);
            res.status(500).send('Error creating user');
        } else {
```

```
            res.status(200).send('User is created successfully');
        }
    });
});
```

Here we have added a new route handler for the /users endpoint that accepts a POST request with a JSON payload containing the name and email address fields for the new user. We will use the pool.query method to execute the SQL INSERT statement to insert a new user into the database. If there is an error, we will return a 500 error response, otherwise, we will return a 200 success response.

- Adding route handler for the Read (SELECT), Delete (DELETE) operations, and Update (UPDATE):

```
// Retrieve all users
app.get('/users', (req, res) => {
    pool.query('SELECT * FROM users', (error, results) => {
        if (error) {
            console.error(error);
            res.status(500).send('Error retrieving users');
        } else {
            res.status(200).json(results);
        }
    });
});
```

```
// Update a user by ID
app.put('/users/:id', (req, res) => {
    const id = req.params.id;
    const { name, emailaddress } = req.body;

    pool.query('UPDATE users SET name = ?, emailaddress = ? WHERE id = ?
        if (error) {
            console.error(error);
            res.status(500).send('Error updating user');
    } else {
        res.status(200).send('User updated successfully');
    }
});

// Delete a user by ID
app.delete('/users/:id', (req, res) => {
    const id = req.params.id;

    pool.query('DELETE FROM users WHERE id = ?', [id], (error, results)
        if (error) {
            console.error(error);
            res.status(500).send('Error deleting user');
    } else {
        res.status(200).send('User deleted successfully');
    }
```

```
    });
});
```

Here we have added the route handlers for the Read, Update, and Delete operations. The user's endpoint will accept a GET request to retrieve all the users, a PUT request to update the user by ID, and a DELETE request to delete the user by ID. We will use the pool.query method to execute SQL statements for each operation, and return appropriate responses based on the success or the failure of each operation.

Now, the CRUD Express.js app is completed. We can start it by running the command in the terminal:

```
node index.js
```

Create Database, Table, and Connection for node.js CRUD Example with MySQL

In the below steps, we will discuss how to create an MYSQL database, and table, and establish a connection to it for a Node.js CRUD application:

- Installing required packages:

```
npm install mysql
```

- Creating an MYSQL database and table:

```
CREATE DATABASE db;

USE db;


CREATE TABLE my_table (

  id INT NOT NULL AUTO_INCREMENT,

  name VARCHAR(255) NOT NULL,

  age INT NOT NULL,

  PRIMARY KEY (id)

);
```

- Establishing a connection to MYSQL in our Node.js application:

```
const mysql = require('mysql');


const connection = mysql.createConnection ({

    host: 'localhost',

    user: coder,

    password: 'password',

    database: 'db’

});


connection.connect((err) => {

  if (err) {

      console.error('Error connecting to MySQL: ', err);

      return;

  }
```

```
    console.log('Connected to MySQL');
});

module.exports = connection;
```

- We can now use this connection object to perform CRUD operations on the my_table table.

  Below is an example of how to fetch all the records from the table.

```
const connection = require('./connection');

connection.query('SELECT * FROM my_table', (err, results) => {
    if (err) {
        console.error('Error retrieving records: ', err);
        return;
    }

    console.log('Retrieved records: ', results);
});
```

Create CRUD Routes

Below are the steps on how to create CRUD Routes in Nodejs using Express and MYSQL.

- Installing the required packages

```
npm install express mysql
```

- Creating a server.js file

```
const express = require('express');
const mysql = require('mysql');

const app = express();

// Create a MySQL connection
const connection = mysql.createConnection ({
    host: 'localhost',
    user: coder,
    password: 'password',
    database: db
});

connection.connect((err) => {
    if (err) {
        console.error('Error connecting to MySQL: ', err);
        return;
    }

    console.log('Connected to MySQL');
});

// Retrieve all records
```

```
app.get('/records', (req, res) => {
    connection.query('SELECT * FROM my_table', (err, results) => {
        if (err) {
            console.error('Error retrieving records: ', err);
            res.status(500).send('Error retrieving records');
            return;
        }

        res.send(results);
    });
});

// Retrieve a specific record
app.get('/records/:id', (req, res) => {
    const id = req.params.id;

    connection.query('SELECT * FROM my_table WHERE id = ?', id, (err, re
        if (err) {
            console.error('Error retrieving record: ', err);
            res.status(500).send('Error retrieving record');
            return;
        }

        res.send(results[0]);
    });
});
```

```
// Create a new record
app.post('/records', (req, res) => {
    const { name, age } = req.body;

    connection.query('INSERT INTO my_table SET ?', { name, age }, (err,
        if (err) {
            console.error('Error creating record: ', err);
            res.status(500).send('Error creating record');
            return;
        }

        res.send(result);
    });
});

// Update an existing record
app.put('/records/:id', (req, res) => {
    const id = req.params.id;
    const { name, age } = req.body;

    connection.query('UPDATE my_table SET name = ?, age = ? WHERE id = ?
        if (err) {
            console.error('Error updating record: ', err);
            res.status(500).send('Error updating record');
            return;
        }
```

```
        res.send(result);
    });
});

// Delete a record
app.delete('/records/:id', (req, res) => {
    const id = req.params.id;

    connection.query('DELETE FROM my_table WHERE id = ?', id, (err, resu
        if (err) {
            console.error('Error deleting record: ', err);
            res.status(500).send('Error deleting record');
            return;
        }

        res.send(result);
    });
});

// Start the server
const PORT = 3000;
app.listen(PORT, () => console.log(`Server started on port ${PORT}`));
```

Run the Express Files

To run the express.js files we need to first save the code in the file named server.js. Then we will start the server with the following command:

```
node server.js
```

This command will start the server at port 3000. We can access the routes we defined in the code by opening a web browser and navigating to http://localhost:3000/.

Deployment

We can deploy the Node js and MYSQL application via the below steps:

- **Choose a hosting provider** – There are many hosting providers such as AWS, Heroku, and Google Cloud Platform which help in performing CRUD in MYSQL with Express.js.
- **Server setup** – Depending on the hosting provider you choose, you may need to create a server instance. You may need to configure your server to meet your application's needs.
- **Upload your code** - You can upload your code to your server using FTP or other file transfer methods. Alternatively, you can deploy your code using a version control tool such as Git.
- **Install dependencies** - After uploading your code, you need to run npm install to install dependencies.

- **Set environment variables** - You may need to set environment variables for your application.

- **Start the server** - Finally, run node server.js or use a process manager like PM2 to start the server to see Express.js performing CRUD in MYSQL.

These are general deployment instructions only. The exact steps may vary depending on your chosen hosting provider and application needs.

Conclusion

- MySQL is used to store and handle huge amounts of data by many major companies and organizations, including Facebook, Twitter, and YouTube.
- CRUD is the collection of fundamental operations that are commonly performed while working with databases while performing CRUD in MYSQL with express.js.
- Node.js and npm are the prerequisites for the express.js in performing CRUD in MYSQL with Express.js.
- There are various platforms on which the application can be deployed. Various factors vary or depend on the platform on which the user chooses to deploy the application.